Lab Week 4 (The 7 Lives of Cats)

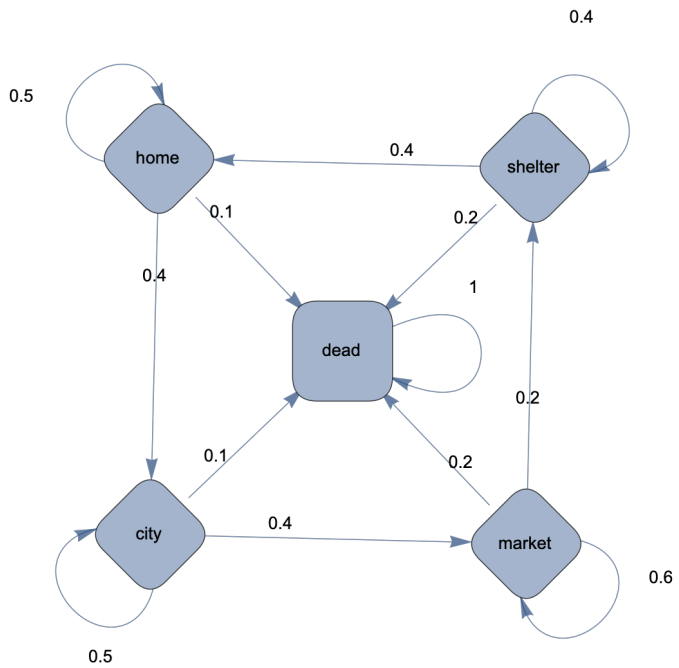
1. The Markov chain states

H=Home (safe but reward not as high as in the market, very low risk of death)
S=Shelter (high likelihood of adoption, small likelihood of dying)
C=(In the) City (only access to Market, high risk being run over, high risk being impounded)
M=Market (high reward, high risk to A, low likelihood to S)
D=Dead

2. Transition Matrix (for the fixed policy expressed in the diagram)

In[ ]:=
$$A = \begin{pmatrix} .5 & 0 & .4 & 0 & .1 \\ .4 & .4 & 0 & 0 & .2 \\ 0 & 0 & .5 & .4 & .1 \\ 0 & .2 & 0 & .6 & .2 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix};$$

Out[ ]=

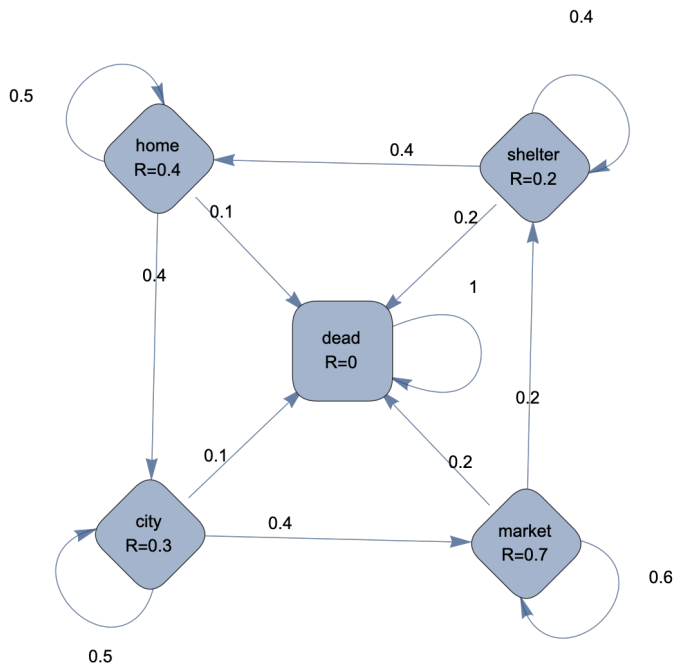


3. As a Reward Process

In[ ]:=
```
rewards = {.4, .2, .3, .7, 0};
```

Display graph with rewards

Out[ ]=



4. Calculating the values in closed form

The next function implements the formula $V = (1 - \gamma P)^{-1} R$

In[ ]:=
```
value[P_, R_, γ_] := Inverse[IdentityMatrix[Length[R]] - γ * P]. R
```

In[ ]:=
```
value[A, rewards, 0.95]
```

Out[ ]=
```
{2.45076, 1.82466, 2.33329, 2.43415, 0.}
```

The same can easily be done in Numpy

In[ ]:=
```
import numpy as np
R=np.array( [.4,.2,.3,.7,0])
P=np.array(
[
[0.5,0.,0.4,0.,0.1],
[0.4,0.4,0.,0.,0.2],
[0.,0.,0.5,0.4,0.1],
[0.,0.2,0.,0.6,0.2],
[0.,0.,0.,0.,1.]
])
gamma=0.95
V=np.matmul(np.linalg.inv(np.identity(R.size)-gamma*P), R)
list(V)
```

Out[ ]=
```
{2.45076, 1.82466, 2.33329, 2.43415, 0.}
```

5. As a Markov Decision Problem: action set

In[ ]:=
```
actions = {stay, run};
```

6. You can easily handle this by assigning the same transition probabilities to either action

7. This can be handled by letting the death state loop deterministically and assigning zero reward to this loop

8. Transition matrix

In[9]:=
$$T = \left\{ \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ .4 & .4 & 0 & 0 & .2 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}, \begin{pmatrix} 0 & 0 & .8 & 0 & .2 \\ .4 & .4 & 0 & 0 & .2 \\ 0 & 0 & 0 & .8 & .2 \\ 0 & .5 & 0 & 0 & .5 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \right\};$$

Out[ ]//TableForm=

	h	s	c	m	d
h	1	0	0	0	0
s	0.4	0.4	0	0	0.2
c	0	0	1	0	0
m	0	0	0	1	0
d	0	0	0	0	1

Out[ ]//TableForm=

	h	s	c	m	d
h	0	0	0.8	0	0.2
s	0.4	0.4	0	0	0.2
c	0	0	0	0.8	0.2
m	0	0.5	0	0	0.5
d	0	0	0	0	1

9. Reward Matrix

In[6]:=
$$R = \begin{pmatrix} .4 & 0 \\ .2 & 0 \\ .3 & 0 \\ .7 & 0 \\ 0 & 0 \end{pmatrix};$$

10. Solving the MDP using Value Iteration in MDPtoolbox

In[ ]:=
```
import numpy as np
```

In[ ]:=
```
import mdptoolbox
```

Note that the notation <* X *> inserts the value for X from above into the Python call (T, R, respectively)

In[ ]:=
```
T=np.array( <* T *> )
```

In[ ]:=
```
R=np.array( <* R *> )
```

In[ ]:=
```
vi = mdptoolbox.mdp.ValueIteration(T, R, 0.95)
vi.run()
```

In[ ]:=
```
vi.V
```

Out[ ]=
```
{8.07992, 5.27444, 10.6319, 13.9899, 0.}
```

In[ ]:=
```
vi.policy
```

Out[ ]=
```
{1, 0, 1, 0, 0}
```

So the computed (deterministic) policy is :

Out[ ]=
```
{home → run, shelter → stay, city → run, market → stay, dead → stay}
```

Performing 1 iteration of Value Iteration manually

First, we need to pick a discount factor γ

In[1]:=
```
γ = 0.95;
```

We initialise the state-value matrix (could be done randomly, here we take {1,2,3,4,5} to make effects visible

In[2]:=
```
V = Range[5]
```

Out[2]=
```
{1, 2, 3, 4, 5}
```

In value iteration, we always use a greedy policy (implicit, we only need to represent the value)

For each value, we pick the action that yields the maximum of (instant reward + expectation of next state value)

Updating State 1 step-by-step

We first look only at action 1

In[19]:=
```
state = 1;
action = 1;
```

The expected reward for action 1 (“stay”) in each state is :

In[7]:=
```
R[[All, action]]
```

Out[7]=
```
{0.4, 0.2, 0.3, 0.7, 0}
```

The transition matrix for action 1 is:

In[10]:=
```
T[[action]] // MatrixForm
```

Out[10]//MatrixForm=

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0.4 & 0.4 & 0 & 0 & 0.2 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

The expectation for the next state value for each state if choosing action 1 is (based on current V):

In[11]:=
```
T[[action]].V
```

Out[11]=
```
{1., 2.2, 3., 4., 5.}
```

The sum of expectation of reward + (discounted) next state value for each state and action 1 then is:

In[21]:=
```
R[[All, action]] + γ T[[action]].V
```

Out[21]=
```
{1.35, 2.29, 3.15, 4.5, 4.75}
```

So, for state 1 it is

In[25]:=
```
(R[[All, action]] + γ T[[action]].V) [[state]]
```

Out[25]=
```
1.35
```

Now we calculate the same for action 2:

In[26]:=
```
action = 2;
```

In[27]:=
```
(R[[All, action]] + γ T[[action]].V)
```

Out[27]=
```
{3.23, 2.09, 3.99, 3.325, 4.75}
```

In[29]:=
```
(R[[All, action]] + γ T[[action]].V) [[state]]
```

Out[29]=
```
3.23
```

Thus, the best action to pick (so far), is action 2 and our new state value (best guess) for state 1 is as in the previous line

Putting it all together

Computing the expected state values for both actions together reduces to :

In[ ]:=
```
T.V // MatrixForm
```

Out[ ]//MatrixForm=

$$\begin{pmatrix} 1. & 2.2 & 3. & 4. & 5. \\ 3.4 & 2.2 & 4.2 & 3.5 & 5. \end{pmatrix}$$

Since the reward matrix is formatted differently, we need to transpose one of these to combine them

In[ ]:=
```
R // MatrixForm
```

Out[ ]//MatrixForm=

$$\begin{pmatrix} 0.4 & 0 \\ 0.2 & 0 \\ 0.3 & 0 \\ 0.7 & 0 \\ 0 & 0 \end{pmatrix}$$

Now we can simply sum R and the expectation of the next state value

In[ ]:=
```
R + γ Transpose[T.V]
```

Out[ ]=
```
{{1.35, 3.23}, {2.29, 2.09}, {3.15, 3.99}, {4.5, 3.325}, {4.75, 4.75}}
```

And finally pick the maximum for each state, wrapping it all into a single line

In[ ]:=
```
Map[Max, R + γ Transpose[T.V]]
```

Out[ ]=
```
{3.23, 2.29, 3.99, 4.5, 4.75}
```

We can now simply iterate this one-line update:

In[ ]:=
```
γ = 0.95;
V = Range[5];
```

In[ ]:=
```
Do[V = Map[Max, R + γ Transpose[T.V]], 1000];
```

In[ ]:=
```
V
```

Out[ ]=

$\{8.0864, 5.27876, 10.64, 14., 2.64591 \times 10^{-22}\}$

If you want to watch this in slow motion, use Monitor[]

In[ ]:=
```
Monitor[
 Do[
  V = Map[Max, R + γ Transpose[T.V]];
  Pause[1], 100
 ],
 V
]
```

In[ ]:=
```
V
```

Out[ ]=
```
{8.05917, 5.27047, 10.5986, 13.9408, 0.0296026}
```

Now the same in Python

First, policy iteration via the mdp toolbox library

In[ ]:=
```
import numpy as np
```

In[ ]:=
```
T=np.array(
    [
```

```
        [
            [1, 0, 0, 0, 0],
            [0.4, 0.4, 0, 0, 0.2],
            [0, 0, 1, 0, 0],
            [0, 0, 0, 1, 0],
            [0, 0, 0, 0, 1]
        ],
        [
            [0, 0, 0.8, 0, 0.2],
            [0.4, 0.4, 0, 0, 0.2],
            [0, 0, 0, 0.8, 0.2],
            [0 , 0.5, 0, 0, 0.5],
            [0, 0, 0, 0, 1]]
    ]
)
```

In[ ]:=

```
R=np.array(
    [
        [0.4, 0]],
        [0.2, 0],
        [0.3, 0],
        [0.7, 0],
        [0.0, 0]
    ]
)
```

In[ ]:=

```
import mdptoolbox
vi = mdptoolbox.mdp.ValueIteration(T, R, 0.95)
vi.run()
vi.V
```

Out[ ]=

{8.07992, 5.27444, 10.6319, 13.9899, 0.}

In[ ]:=

```
vi.policy
```

Out[ ]=

{1, 0, 1, 0, 0}

Now, manually (as above in Mathematica)

In[ ]:=

```
gamma=0.95
```

In[ ]:=

```
V=np.array(range(1,6))
```

In[ ]:=

```
np.max(R+gamma*np.transpose(np.matmul(T,V)),axis=1)
```

Out[ ]=

NumericArray[Type: Real64
Dimensions: {5}
Data: 3.23, 2.29, 3.99, 4.5, 4.75]

In[ ]:=

```
for i in range(100):
   V=np.max(R+gamma*np.transpose(np.matmul(T,V)),axis=1)
```

In[ ]:=

```
V
```

Out[ ]=

NumericArray [Type: Real64
Dimensions: {5}
Data: 8.06, 5.27, 10.6, 13.9, 0.0296]